

Tema 4 — Estructuras de datos concurrentes y lock-free

Fuentes: Teóricas 04-Listas y 05-colas, Práctica clase-04 (listas y skip lists), Guía 4, Labo 3. **Ejercicio 4 del parcial.**

1. ¿Qué significa que una estructura concurrente sea "correcta"? (safety)

En secuencial, una operación es un instante: la especificación dice cómo está el objeto antes y después. En concurrente, cada llamada **dura un intervalo** (de la invocación a la respuesta) y los intervalos se pisan entre sí.

- **Consistencia secuencial:** existe *algún* orden secuencial de todas las llamadas que respeta el orden de cada thread y cumple la especificación.
 - **No respeta el tiempo real** entre threads distintos.
 - **No es composicional:** dos colas que cada una es SC pueden formar juntas un sistema que no lo es.
- **Linealizabilidad** ★: cada llamada parece ocurrir **en un instante** (su **punto de linealización**, PL) **dentro** de su intervalo.
 - Respeta el tiempo real y **sí es composicional**.
 - Toda ejecución linealizable es SC; al revés no.
 - **Cómo se justifica en el parcial:** para cada operación (y para cada caso: exitosa o no) indicás el PL, que es un paso atómico. Después mostrás que, ordenando por PL, la historia cumple la especificación secuencial.
 - **Guía de PLs:**
 - Con locks: cualquier instante dentro de la sección crítica, típicamente la escritura.
 - Sin locks: el **CAS exitoso** (o la lectura que decide el resultado).

En criollo: es como si cada operación, en algún momento entre que la llamaste y que te respondió, pasara de golpe, en un "clic". Si podés ubicar ese clic para todas las operaciones y el resultado tiene sentido como si fueran de a una, la estructura es linealizable.

2. ¿Qué garantiza que avance? (progreso)

Garantía	Qué asegura	Tipo
Wait-free	toda llamada termina en una cantidad finita de pasos, haga lo que haga el resto	no bloqueante (la mejor)
Lock-free	alguna llamada siempre termina (progreso global; alguien puede sufrir inanición)	no bloqueante
Starvation-free	toda llamada termina, <i>si el scheduler es fair</i>	bloqueante (con locks)
Deadlock-free	alguna llamada termina, <i>si el scheduler es fair</i>	bloqueante

Wait-free $\Rightarrow$ lock-free. Con locks, lo mejor que podés tener es starvation-free, y eso **solo si los locks son fair**.

3. CAS: la herramienta del lock-free

`compareAndSet(esperado, nuevo)`: "si vale `esperado`, ponele `nuevo`", atómicamente. Devuelve si lo logró.

Patrón **CAS loop**:

```
do { v = x.get(); } while (!x.compareAndSet(v, v + 1));
```

- Es **lock-free**: si tu CAS falló, es porque **otro** tuvo éxito.
- **No es wait-free**: te pueden ganar infinitas veces.

4. Conjuntos con listas enlazadas (teórica 4 + labo 3) ★

La lista está ordenada por clave (el hash), con dos centinelas: `head` = $-\infty$ y `tail` = $+\infty$. Para modificar necesitás `pred` (el nodo anterior) y `curr` (el nodo en cuestión).

Versión	Idea en criollo	contains	Progreso
Gruesa	un solo candado para toda la lista (<code>synchronized</code>)	con lock	deadlock-free; sin inanición si el lock es fair
Fina (hand-over-hand)	cada nodo tiene su candado; avanzás "de mano en mano": tomás el siguiente antes de soltar el anterior. Siempre tenés tomados <code>pred</code> y <code>curr</code>	con locks	deadlock-free (todos toman en orden <code>head</code> → <code>tail</code>); sin inanición si los locks son fair
Optimista	recorrés sin locks; tomás <code>pred</code> y <code>curr</code> ; validás recorriendo de nuevo desde <code>head</code> que <code>pred</code> sigue siendo alcanzable y que <code>pred.next == curr</code> ; si no, reintentás	con locks + validación	deadlock-free; puede haber inanición aunque los locks sean fair (siempre te cambian la lista)
Lazy	cada nodo tiene <code>marked</code> . Borrar = primero marcar (borrado lógico) y después desenganchar. <code>validate = !pred.marked && !curr.marked && pred.next == curr</code> , sin recorrer de nuevo	wait-free , sin locks	<code>add/remove</code> pueden sufrir inanición
Lock-free	sin locks. <code>next</code> y <code>marked</code> son una sola unidad (<code>AtomicMarkableReference</code>), así que el CAS falla si el nodo está marcado. <code>find()</code> va limpiando los nodos marcados que encuentra en el camino	wait-free	<code>add/remove</code> lock-free

Detalles que se preguntan:

- **¿Por qué en la versión fina hay que tomar también el lock del nodo que borro?** Si solo tomás el de `pred`, otro thread puede estar borrando `curr` al mismo tiempo, y uno de los dos borrados se pierde.
- **Orden de los locks**: siempre `pred` antes que `curr`. Si en algún lado cambiás el orden $\rightarrow$ deadlock (Guía 4 ej 4b).
- **PLs (lazy)**:
 - `add` exitoso: `pred.next = nuevo`;

- remove exitoso: `marked = true;`
- contains exitoso: cuando encuentra el nodo sin marcar;
- contains que falla: es sutil, puede estar justo antes de un add concurrente.
- **PLs (lock-free):**
 - add: el CAS que engancha el nodo;
 - remove: el CAS que **marca**.

Skip lists (práctica 4, prioridad baja)

- Una lista con varios niveles de "atajos" que simula un árbol. Búsqueda esperada $O(\log n)$, sin las rotaciones de un AVL (que generan contención).
- El nivel de cada nodo es aleatorio, con probabilidad p^i .
- Hay versión **Lazy** (locks + `marked` + `fullyLinked`) y versión **LockFree**. En las dos, `contains` es wait-free.

5. Colas y pilas (teórica 5)

- **Cola acotada con dos locks** (`enqLock` para el tail, `deqLock` para el head) + `size` atómico: un productor y un consumidor pueden trabajar al mismo tiempo.
 - Solo se despierta al otro bando en las transiciones "estaba vacía" / "estaba llena", y **tomando el lock del otro bando**, para que no se pierda el wake-up.
 - `size` puede quedar en -1 un instante.
 - PL del enq: `tail.next = e`.
- **Cola no acotada total:** si está vacía, `deq` tira excepción en vez de esperar.
- **Cola lock-free (Michael-Scott):**
 - `enq` hace CAS de `last.next` y después intenta avanzar `tail`.
 - **Helping:** si alguien ve el `tail` atrasado, lo avanza él, y así nadie queda trabado esperando a un thread lento.
 - PL: el CAS de `last.next` (`enq`) y el CAS de `head` (`deq`).
- **Pila lock-free:** CAS sobre `top`. **Backoff exponencial** cuando falla, para bajar la contención.
- **Problema ABA** ★ (en C, o si reciclás nodos a mano sin garbage collector):
 - Leíste `head = A` y te dormiste.
 - Mientras tanto sacaron A y B y **volvieron a meter el mismo nodo A**.
 - Tu CAS "¿head sigue siendo A?" da verdadero, y engancha basura.
 - **Solución:** `AtomicStampedReference`, un sello o versión que se incrementa en cada cambio. Aunque la dirección sea la misma A, el sello ya no coincide.
- **Eliminación:** un push y un pop que se cruzan se cancelan entre sí sin tocar la pila (con un `Exchanger`, un arreglo de intercambiadores). Prioridad baja.
- **Cola síncrona / rendezvous:** el que pone espera a que alguien lo saque. Estructuras duales. Prioridad baja.

6. Recetas para el Ej 4 del parcial

- **"Granularidad fina" sobre algo con N partes:**
 - un lock por parte;
 - las operaciones sobre varias partes **toman los locks en orden creciente** (no hay deadlock);
 - las operaciones sobre "todo" (una suma) toman todos en orden, así que hay un snapshot real y el PL es cuando tiene todos tomados.

- **"Optimista"**: leer, computar sin lock, tomar el lock, validar (o comparar versión), escribir o reintentar. Sin inanición: **no**.
- **"Lock-free con CAS"**: el CAS loop. Para modificar varios campos juntos: un objeto **immutable** en un `AtomicReference` y CAS del objeto entero (Figura, Guía 4 ej 11).
- **Double collect** (leer todo dos veces y comparar):
 - Si los valores solo crecen, que las sumas coincidan implica que no cambió nada → es linealizable.
 - Con decrementos o `reset` no alcanza comparar sumas: hay que comparar arreglo contra arreglo, y por ABA hacen falta versiones.
 - No es wait-free: puede repetir infinitamente.

7. Guía 4 — qué patrón entrena cada ejercicio

Ej	Patrón
1	listas con claves repetidas: qué cambia en cada versión
2	Figura: un lock por grupo de campos + orden fijo si tocás los dos
3	Recursos <code>swap(i, j)</code> : lock por posición, tomar min primero; ¿inanición?
4	optimista: inanición (a) y orden de locks invertido (b)
5	<code>actualizarEntrada</code> optimista (computar fuera del lock + validar)
6	cola con dos pilas: la race de <code>deq</code> y cómo arreglarla
7	cola acotada con dos contadores (menos contención sobre <code>size</code>)
8	pila acotada
9	ABA en la pila lock-free + sellos
10	contador lock-free <code>inc</code> / <code>get</code> / <code>reset</code> : ¿cuáles son wait-free?
11	Figura lock-free (objeto immutable + CAS)